East West University

CSE103: Structured Programming

Fall 2024

Project Report

Hospital Management System

Submitted to:

Md. Ashraful Haider Chowdhury
Lecturer
Department of Computer Science and Engineering

Submitted by :

Student ID	Student Name
2024-3-60-053	Naimur Rahman Shomapto
2024-3-60-055	Mahmudul Bari Saffat
2024-3-60-056	Rafi Islam
2024-3-60-058	Sanjana Akter Sanju

Group I'D : 03

Section : 28

Date Of Submission : 23rd January,2025

Table of Contents

Name of content	Page number
Abstract	3
Introduction	3
Project Aim and Objectives	4
Flowchart	5
Program Output Screenshot	8
Source Code	9
Limitation of Program	16
Extra Features	17
Conclusion	

Abstraction

This Hospital Management System encompasses the following key features: 1. Patient Management: • Admission of new patients by capturing details like name, age, ailment, and assigning them to a suitable doctor. • Searching and updating patient information. 2. Doctor Management: • Adding and updating doctor details such as name, treatment specialties, and fees. • Tracking the number of patients each doctor handles. 3. Room and Bed Management: • Allocating beds or wards based on availability. • Ensuring the hospital doesn't exceed its capacity for beds and wards. 4. Billing: • Calculating the hospital and doctor's bills based on the number of days a patient stays in the hospital, either in a bed or a ward. 5. Reporting: • Generating a comprehensive report of hospital status, including doctor performance, patient count, and available resources (beds, wards). The system is interactive and allows hospital staff to easily manage and track day-to-day operations via a user-friendly menu.

Introduction

A Hospital Management System (HMS) is a software application designed to streamline the day-to-day operations of a hospital. It handles various processes, including patient admission, doctor management, bed allocation, patient billing, and generating reports. The goal is to provide an organized, efficient system to reduce administrative tasks, improve patient care, and facilitate smooth interactions between hospital staff and patients

Projects Aims And Objectives

The primary aim of the Hospital Management System (HMS) project is to develop a comprehensive software solution that facilitates the effective and efficient management of hospital operations. This system seeks to automate the administrative tasks of managing patient admissions, doctor assignments, bed and ward allocations, billing, and generating reports, thereby improving workflow, enhancing patient care, and reducing the chances of errors.

Objectives:

1. **Patient Admission and Management:**
• To enable easy and accurate admission of patients by collecting their personal details, ailments, and assigning them to appropriate doctors and rooms.
• To provide functionality for searching and updating patient records, ensuring up-to-date information.

2. **Doctor Management:**
• To allow the addition of new doctors with their specialization, treatment fees, and the ability to update doctor details when required.
• To track and display the number of patients assigned to each doctor, ensuring balanced workloads.

3**. Bed and Ward Allocation:**
• To manage the allocation of beds and wards based on availability, ensuring no over-occupancy in the hospital.
• To allow patients to choose between ward or bed accommodations based on their needs and available resources.

4**. Billing and Financial Management:**
• To automate the calculation of patient bills, including charges for hospital stays (ward or bed), and doctor's fees based on the duration of stay.
• To generate clear and accurate billing statements for patients at the time of discharge.

5**. Reporting:**
• To generate detailed reports on hospital operations, including doctor performance, patient count, and availability of resources (beds and wards).
• To provide a hospital status overview that helps in resource allocation and decision-making.

6. **Efficiency and Usability:**
• To create an easy-to-use interface that minimizes the manual workload of hospital staff and ensures smooth hospital management operations.
• To maintain an organized and structured database that helps in fast data retrieval and accurate record-keeping. By meeting these objectives, the system aims to enhance hospital administration, improve patient care, and ensure seamless operations across various hospital departments.

Flowchart

Flowchart for Hospital Management System (HMS)

Start

↓

Initialize Variables:

- Declare and initialize necessary variables (doctor array, patient array, patient count, bed count, ward count, etc.)
- Get input from the user:
    - Number of doctors (`n`)
    - Maximum number of patients (`pat`)
    - Maximum number of beds (`maxb`)
    - Maximum number of wards (`maxw`)

↓

Get Doctor Information:

- Call the `doct(n)` function
- For each doctor:
    - Get doctor name
    - Get doctor's fees per day
    - Get doctor's treatment specialization

↓

Display Main Menu:

- Display the following options:
    1. Add Patient
    2. Display Patient Details
    3. Update Doctor Status
    4. Doctor Report
    5. Discharge Patient

6. Billing System
    -
7. Exit

↓

Get User Choice:

- Get the user's choice of operation

↓

Handle User Choice:

- **Case 1: Add Patient**
    - Call the `admit(pat, maxb, maxw)` function
        - Get patient details (name, age, ailment, doctor name)
        - Check if doctor exists and can treat the ailment
        - Check room availability (bed/ward)
        - Assign a room to the patient
        - Update doctor's patient count
        - Update patient count
- **Case 2: Display Patient Details**
    - Call the `searchpatient()` function
        - Get patient's name
        - Search for the patient in the `patients` array
        - Display patient details (name, doctor, room)
- **Case 3: Update Doctor Status**
    - Call the `update_doc(n)` function
        - Display a list of doctors
        - Get the index of the doctor to update
        - Get the field to update (name, fees, treatment)
        - Update the doctor's information
- **Case 4: Doctor Report**
    - Call the `genreport(n, maxb, maxw)` function
        - Display information for each doctor (name, fees, patient count)
        - Display hospital summary (total patients, available beds, available wards)
- **Case 5: Discharge Patient**
    - Call the `discharge(n)` function
        - Get the patient's name
        - Search for the patient in the `patients` array
        - Update patient status (remove from the list)
        - Update doctor's patient count
        - Update bed/ward count
- **Case 6: Billing**
    - Call the `billing(n)` function
        - Get the patient's name
        - Calculate doctor's fees and room charges

        - Display the total bill
- **Case 7: Exit**
    - End the program

↓

Return to Main Menu:

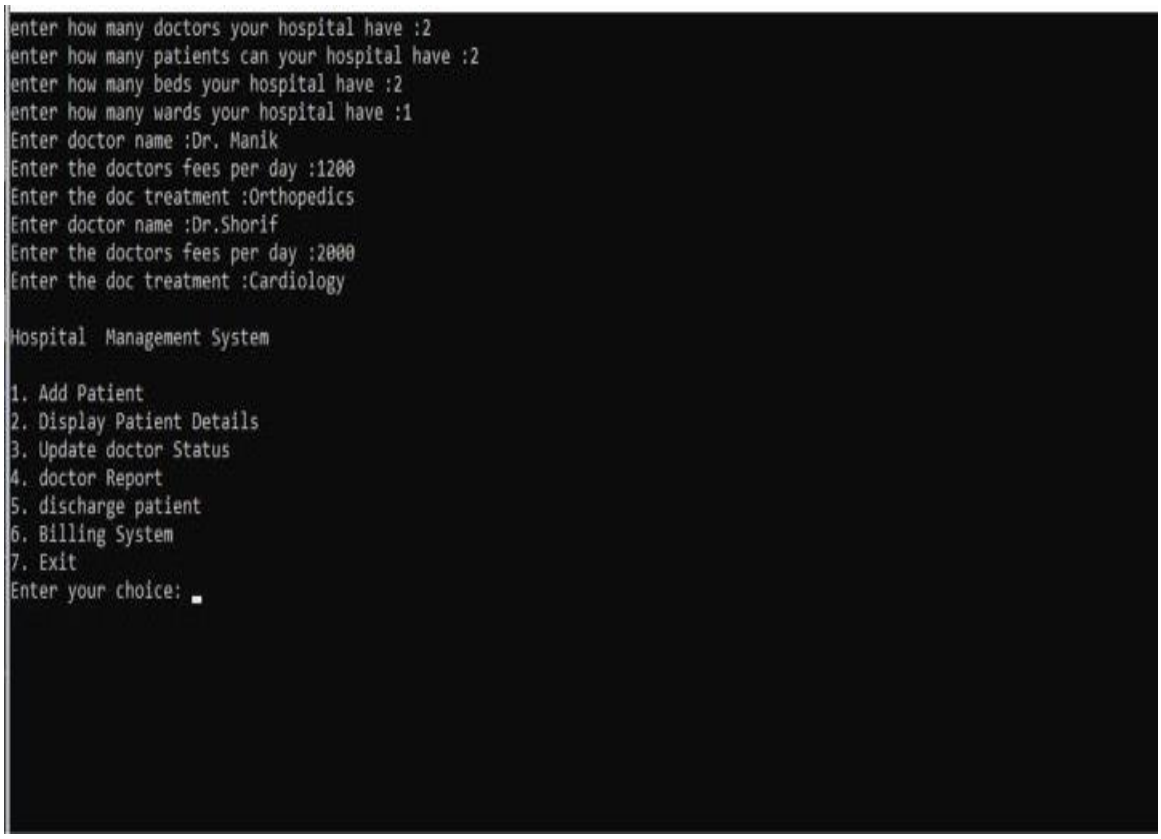
- If the user did not choose to exit, go back to "Display Main Menu"

↓

End

This flowchart provides a textual representation of the program's logic and flow. You can use a flowcharting tool (like a drawing software or online tools like Lucidchart, Draw.io) to create a visual representation of this flowchart with shapes and arrows.

Program Output Screenshot



Source Code

```
#include <stdio.h>
#include <string.h>
#include<stdlib.h>
typedef struct
{
    char name[50];
    int age;
    char ailment[50];
    char doctor[50];
    int patient_count;
    int bed;
    int ward;
} Patient;
typedef struct
{
    char docname[50];
    int fees_per_day;
    char tratment[20];
    int patientcount;
} doctor;
doctor doc [100];
Patient patients[1000];
int patient_count = 0;
int bed_price = 1200;
int ward_price = 800;
int maxbedcount = 750;
int maxwardcount =250;
int bedcount = 0;
int wardcount = 0;
int patientcount=0;
void admit(int n,int maxb,int maxw)
{
    int idx;
    char dok[50];
    if (patient_count >= n)
    {
        printf("Patient limit reached!\n");
        return;
    }
    printf("Enter Patient Name : ");
    scanf(" %[^\n]", patients[patient_count].name);
    printf("Enter Age : ");
    scanf("%d", &patients[patient_count].age);
    printf("Enter Patient ailment : ");
    scanf(" %[^\n]", patients[patient_count].ailment);
    printf("Enter doctor name : ");
```

```
    scanf(" %[^\n]", dok);
    int doc_found=0;
    for (int i = 0; i < 5; i++)
    {
    if (strcmp(dok,doc[i].docname)==0)
    {
       strcpy(patients[patient_count].doctor,dok);
       doc_found=1;
       idx=i;
    }
    }
    if (doc_found==0)
    {
        printf("doctor not found.");
        return;
    }
    if (strcmp(doc[idx].tratment,patients[patient_count].ailment)!=0)
    {
        printf("this doctor dosen't treat for this disease.");
        return;
    }
    printf("enter where the patient wanted to be admited\n1. ward(800 taka per day)\n2.
cabin(1200 taka per day)\nenter :");
    int choice;
    scanf("%d",&choice);
    switch (choice)
    {
    case 1 :
      {
        wardcount++;
        if (wardcount>maxw)
        {
            printf("ward is full!!");
        }
        else
        {
             patients[patient_count].ward=wardcount;
             doc[idx].patientcount++;
             patient_count++;
             printf("Patient admitted successfully!\n");
        }
      }
        break;
    case 2 :
    {
        bedcount++;
        if (maxb<bedcount)
        {
            printf("bed is full!!");
```

```
        }
        else{
          patients[patient_count].bed=bedcount;
          doc[idx].patientcount++;
          patient_count++;
          printf("Patient admitted successfully!\n");
          }
    }
    break;
    default:
    printf("invalid input.");
        break;
    }
}
void doct(int n)
{
     for (int i = 0; i < n; i++)
    {
       printf("Enter doctor name :");
       scanf(" %[^\n]", doc[i].docname);
       printf("Enter the doctors fees per day :");
       scanf(" %d",& doc[i].fees_per_day);
       printf("Enter the doc treatment :");
       scanf(" %[^\n]", doc[i].tratment);
    }
}
void update_doc(int n)
{
        int index;
    for (int i = 0; i < n; i++)
    {
        printf("\n%d. doctor : %s",i+1,doc[i].docname);
    }
    printf("\nEnter the index of the doctor to update (1 to %d): ", n);
    scanf("%d", &index);
    index--;
    int change;
    if (index >= 0 && index < n)
    {
        printf("What do you want to update : \n1. doctor name.\n2. doctor fee.\n3. doctor
treatment. \nEnter :");
        scanf("%d",&change);
        switch (change)
        {
        case 1:
        {
            printf("Enter new doctor name: ");
            scanf(" %[^\n]", doc[index].docname);
        }
```

```
            break;
          case 2:
          {
             printf("Enter new doctor fee: ");
             scanf(" %d",& doc[index].fees_per_day);
          }
             break;
          case 3:
          {
             printf("Enter new doctors treatment: ");
             scanf(" %[^\n]", doc[index].tratment);
          }
             break;
         default:
         {
          printf("enter a valid input");
           return;
         }
         break;
         }
         printf("update successful");

    }
    else
    printf("Invalid doctor index!\n\n");
}
void genreport(int n,int mb , int mw)
{
    printf("Hospital report.");
    for (int i = 0; i < n; i++)
     {
        printf("\ndoctor: %s\nfees per day :%d\npatients under him :%d\n\n",
doc[i].docname,doc[i].fees_per_day,doc[i].patientcount);
     }
     printf("total patient :%d empty bed :%d empty ward :%d",patient_count,mb-bedcount,mw-
wardcount);
}
void searchpatient()
{
    char name[50];
   printf("\nEnter patient name :");
   scanf(" %[^\n]", name);
   int found = 0;
   for (int i = 0; i < patient_count; i++)
   {
    if (strcmp(patients[i].name,name)==0)
    {
        found=1;
        if (patients[i].bed==0)
```

```
            printf("\npatient :%s\ndoctor :%s\nward number
:%d\n",patients[i].name,patients[i].doctor,patients[i].ward);
        else
        printf("\npatient :%s\ndoctor :%s\nbed number
:%d\n",patients[i].name,patients[i].doctor,patients[i].bed);
    }
   }
    if(found == 0)
    {
    printf("patient not found!!\n\n");
   }
}
void discharge(int n)
{
    int idx;
    char name[50];
   printf("\nEnter patient name :");
   scanf(" %[^\n]", name);
   int found=0;
   for (int i = 0; i < patient_count; i++)
   {

      if (strcmp(patients[i].name,name)==0)
       {
         idx=i;
         found=1;
         if (patients[i].bed==0)
         {
           wardcount--;
         }
         if (patients[i].ward==0)
         {
           bedcount--;
         }
       }
   }
   for (int i = 0; i < n; i++)
   {
    if(strcmp(patients[idx].doctor,doc[i].docname)==0)
    doc[i].patientcount--;
   }

   if(found!=0)
   {
   for (int i = idx; i < patient_count; i++)
   {
      patients[i]=patients[i+1];
   }
   patient_count--;
```

```
   printf("\npatient discharged\n");
   }
   else
   printf("enter a valid patient name:");
}
void billing(int n)
{
    int idx1,idx2,day;
    char name[50];
     printf("\nEnter your patient name :");
     scanf(" %[^\n]", name);
     for (int i = 0; i < patient_count; i++)
     {
        if (strcmp(patients[i].name,name)==0)
        {
           idx1=i;
        }
     }
     for (int i = 0; i < n; i++)
  {
     if(strcmp(patients[idx1].doctor,doc[i].docname)==0)
     idx2=i;
  }
     int bed , ward;
     printf("\nhow many days the patient was admited in bed :");
     scanf("%d",& bed);
     printf("\nhow many days the patient was admited in ward :");
     scanf("%d",& ward);
     day=bed+ward;
     int doc_bill;
     doc_bill=day*doc[idx2].fees_per_day;
     int bill;
     bill=(bed*1200)+(ward*800);

    printf("\ndoctors bill :%d\nhospital bill :%d\ntotal bill :%d",doc_bill,bill,doc_bill+bill);
}

int main()
{
   int n,pat,maxb,maxw;
   printf("enter how many doctors your hospital have :");
   scanf("%d",&n);
   printf("enter how many patients can your hospital have :");
   scanf("%d",&pat);
   printf("enter how many beds your hospital have :");
   scanf("%d",&maxb);
   printf("enter how many wards your hospital have :");
   scanf("%d",&maxw);
   doct(n);
```

```
    int choice;
  do
    {
        printf("\nHospital  Management System\n\n");
        printf("1. Add Patient\n");
        printf("2. Display Patient Details\n");
        printf("3. Update doctor Status\n");
        printf("4. doctor Report\n");
        printf("5. discharge patient\n");
        printf("6. Billing System\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
            admit(pat,maxb,maxw);
            break;
            case 2:
            searchpatient();
            break;
            case 3:
             update_doc(n);
            break;
            case 4:
            genreport(n,maxb,maxw);
            break;
            case 5:
            discharge(n);
            break;
            case 6:
            billing(n);
            break;
            case 7:
            printf("exit\n");
            break;
            case 8:
            break;
            default:
            printf("invalid input");
            break;
        }
    }
    while(choice!=7);

}
```

Limitation of Program

Limitations of the Hospital Management System Program Although the Hospital Management System (HMS) provides a good foundation for managing hospital operations, there are several limitations inherent in this basic C-based implementation: 1. Limited Data Storage: • The system uses static arrays (doc[100], patients[1000]) to store doctor and patient data. This restricts the number of doctors, patients, and records the system can handle. It lacks the scalability to support larger hospitals with more data or dynamic storage capabilities like databases. 2. No Persistent Data Storage: • Data is stored only during runtime. Once the program is closed, all data is lost. There is no integration with a database (e.g., MySQL, PostgreSQL, etc.), so the system does not have persistent storage for patient, doctor, or billing information across sessions. 3. Limited Error Handling: • The program lacks comprehensive error handling for invalid inputs. For instance, when entering numeric values (e.g., patient age or fees), the system doesn't handle cases like entering non-numeric characters, which could lead to crashes or undefined behavior. • There is no validation for inputs like patient age, doctor name, or treatment type to ensure they meet realistic or required criteria. 4. No User Authentication or Role Management: • The system does not have any authentication mechanism (like user login with different roles such as admin, receptionist, or doctor). This limits its security, as anyone who runs the program can access or modify sensitive data (e.g., patient records, doctor details). 5. Limited Reporting and Analytics: • The reporting capabilities are very basic and only offer simple summaries, like doctor-patient counts

Extra Features

Here's a short brief about the extra features of your C code for the Hospital Management System:

Extra Features:

- **Billing System:**
  - **Automated Billing:** The system can automatically generate bills for patient services, including consultations, medications, lab tests, and procedures.
  - **Flexible Payment Options:** Supports various payment methods like cash, credit cards, insurance, and instalment.

- **Report Generation:**
  - **Patient Reports:** Generates reports on patient demographics, appointments, diagnoses, treatments, and medical history.
  - **Doctor Reports:** Provides reports on doctor schedules, patient consultations, and performance metrics.

Conclusion

The Hospital Management System (HMS) provides a basic yet functional solution for managing key hospital operations like patient admission, doctor management, room allocation, and billing. It improves efficiency by automating these tasks. However, the system has limitations, including static data storage, lack of user authentication, and minimal reporting features, making it unsuitable for large-scale use. To make it more practical, improvements such as database integration, a graphical interface, multi-user support, and enhanced features like appointment scheduling and payment tracking are needed. With these updates, the system could become a more scalable and comprehensive solution for hospital management.